FPT UNIVERSITY / EMBEDDED LINUX SYSTEMS

BÁO CÁO THỰC HÀNH

LAB–02

DEVICE DRIVER & QUẢN LÝ FILE SYSTEM

Character device · procfs/sysfs · MTD/JFFS2 · tự nạp module khi boot

THÀNH VIÊN THỰC HIỆN

HỌ VÀ TÊN	MÃ SỐ SINH VIÊN
Lê Tiến Thành	SE201799
Lê Tấn Lộc	SE201412
Nguyễn Quang Huy Đạt	SE201415
Trần Đình Khôi Nguyên	SE200558
Trần Trương Tuấn Vũ	SE196467

Nền tảng: Ubuntu (HOST) • ARMv7 / QEMU vexpress-a9 (TARGET)

Thực nghiệm: 09–10/2026 | Tổng hợp: 21/10/2026

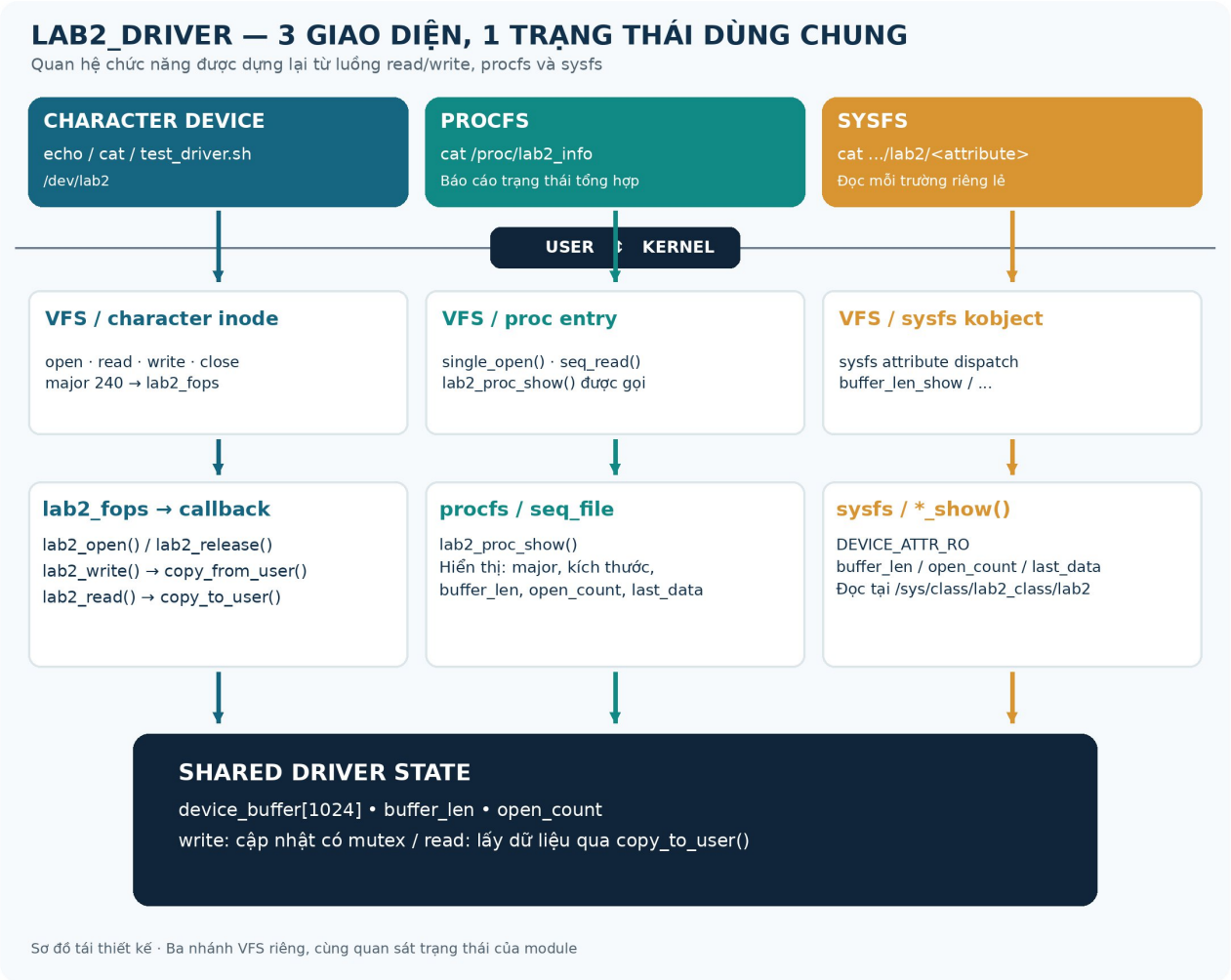
1. Mục tiêu, phân vùng thực nghiệm

Lab 2 nối tiếp Lab 1 bằng một module kernel ARMv7 có giao diện character device /dev/lab2. Ngoài read/write, nhóm trình bày thống kê qua procfs và sysfs, sau đó khảo sát JFFS2 trên NAND giả lập. Cuối cùng module được nhúng vào initramfs để BusyBox init tự tải trong quá trình boot.

Môi trường	Công việc thực hiện	Đặc điểm ghi nhận
TARGET – QEMU ARMv7	insmod lab2_driver, test read/write, procfs/sysfs, rcS	Linux 5.15.0, vexpress-a9, /dev/lab2 major 240
HOST – Ubuntu x86_64	nandsim, mtd-utils, mkfs.jffs2, mount/remount	Kernel host 6.8.0-147-generic; NAND 16 MiB
Toolchain & initramfs	Cross-compile arm-linux-gnueabihf-; nhúng .ko	Module ARM ELF32; vermagic 5.15.0 SMP mod_unload ARMv7 p2v8

Phân biệt HOST/TARGET là yêu cầu an toàn: các thao tác xóa flash như flash_erase phải chạy đúng trên nandsim được nhận diện qua /proc/mtd, không được mặc định tin rằng NAND giả lập luôn có số thiết bị mtd0 hoặc mtd1.

2. Kiến trúc lab2_driver và luồng tương tác



Hình 1. Sơ đồ kiến trúc tái thiết kế: ba giao diện userspace độc lập cùng tham chiếu trạng thái driver.

2.1 Tại sao ba nhánh phải tách biệt?

Nhánh character device đi qua inode /dev/lab2 rồi tra bảng lab2_fops của driver. Hai nhánh procfs và sysfs cũng đi qua VFS, nhưng sử dụng proc entry và device kobject của riêng chúng, không gọi read/write callback của /dev/lab2. Cả ba cuối cùng quan sát cùng device_buffer[1024], buffer_len và open_count.

2.2 Trình tự data path với write và read

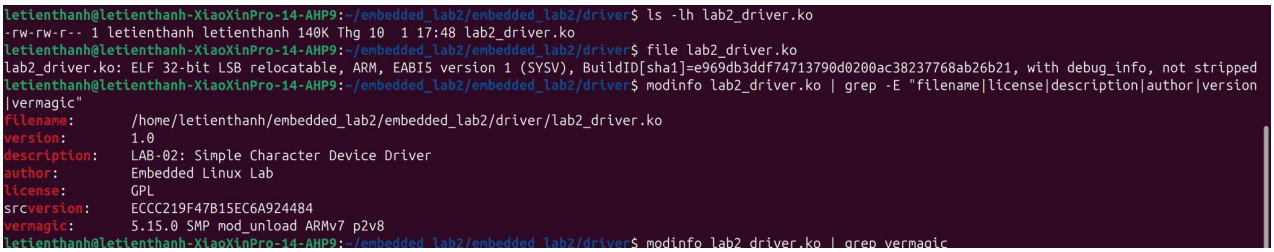
```
USER write(fd, buf, n)
  → VFS → lab2_write() → copy_from_user() → device_buffer
  → buffer_len được cập nhật; file /proc và /sys phản ánh trạng thái
USER read(fd, buf, n)
  → VFS → lab2_read() → copy_to_user() → buffer userspace
```

Mutex dùng để bảo vệ vùng cập nhật chia sẻ, đặc biệt trong đường ghi. Tuy nhiên, mức độ đồng bộ toàn diện giữa callback đọc và *_show() cần được đối chiếu source driver; thành công của các test tuần tự không có nghĩa đã kiểm thử race condition khi nhiều tiến trình thao tác đồng thời.

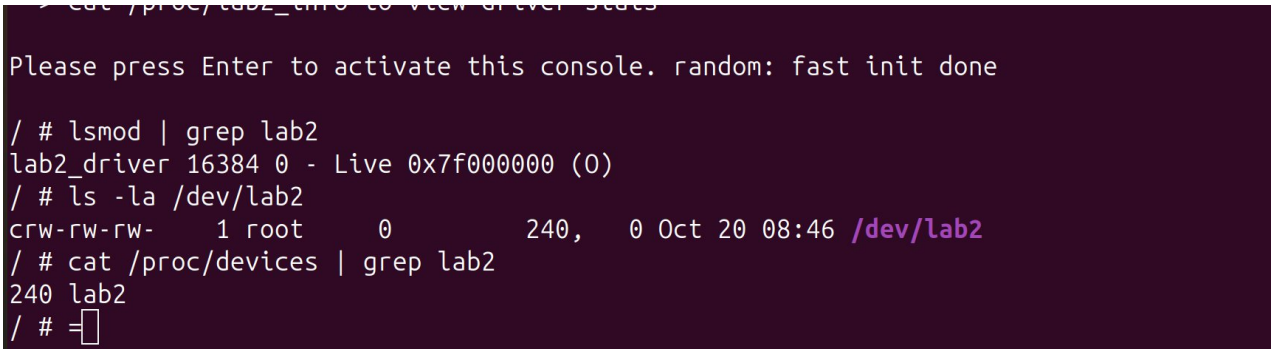
3. Phần A — Module và Character Device

3.1 Tính tương thích của module ARM

KBuild tạo lab2_driver.ko ở dạng ELF32 relocatable cho ARM. modinfo xuất vermagic 5.15.0 SMP mod_unload ARMv7 p2v8 phù hợp kernel target của bài. Sau insmod, /proc/devices và device node đồng thời ghi nhận major 240, minor 0; con số major này được quan sát ở lần chạy, không cần hard-code trong mọi hệ thống.



Hình 2. Kết quả file/modinfo xác định module ARM và phiên bản kernel.



Hình 3. Module đã nạp, /dev/lab2 đăng ký major 240 trên QEMU.

3.2 Đọc/ghi thực tế qua /dev/lab2

test_driver.sh kiểm tra khả năng mở thiết bị, ghi một chuỗi vào kernel buffer và đọc lại từ userspace. Ba lượt Message_1, Message_2, Message_3 được lặp lại và kết thúc Test PASSED. Dấu xuống dòng do echo thêm vào cũng là một byte dữ liệu nên số byte nhận được có thể lớn hơn số ký tự nhìn thấy.



Hình 4. Bài test read/write thực hiện trên character device.



Hình 5. Kết thúc chuỗi Message_1/2/3: Test PASSED.

open_count là bộ đếm số lần lab2_open() được gọi kể từ khi load module, không phải số file descriptor hiện còn mở. Chuỗi log opened/received/sent/closed cho phép đối chiếu callback khi chạy script.

4. Phần B — Cung cấp thông tin qua procfs và sysfs

4.1 procfs: bản thống kê trong một file

Entry /proc/lab2_info dùng seq_file để xuất nhiều trường: tên driver, major, buffer size, data length, open count và last data. Ở bài kiểm thử của nhóm, trước khi ghi data length là 0; sau khi ghi “Embedded Linux Lab2 Test Data” giá trị trở thành 30 bytes, đồng thời nội dung last data được cập nhật.

```
/ # echo "===== RUN TEST_PROCFS.SH ====="
===== RUN TEST_PROCFS.SH =====
/ # test_procfs.sh
=== Test procfs & sysfs ===
lab2_driver: initializing module
lab2_driver: loaded, major=240
lab2_driver: /proc/lab2_info created
--- /proc/lab2_info (before write) ---
LAB-02 Driver Statistics
=====================
Driver name : lab2
Major number : 240
Buffer size : 1024 bytes
Data length : 0 bytes
Open count : 0
lab2_driver: device opened (count=1)
lab2_driver: received 30 bytes: [Embedded Linux Lab2 Test Data
]
lab2_driver: device closed
--- /proc/lab2_info (after write) ---
LAB-02 Driver Statistics
=====================
Driver name : lab2
Major number : 240
Buffer size : 1024 bytes
Data length : 30 bytes
Open count : 1
Last data : [Embedded Linux Lab2 Test Data
]
```

Hình 6. procfs trước/sau write; giá trị thay đổi đúng theo dữ liệu đã ghi.

4.2 sysfs: các attribute của device

Sysfs gắn dữ liệu với device kobject trong /sys/class/lab2_class/lab2. Driver tạo ba thuộc tính read-only buffer_len, open_count và last_data; sysfs core còn có thể cấp thêm những file chuẩn như dev, uevent hoặc các thư mục power, subsystem. Đọc từng attribute phù hợp với script vì không phải phân tích nhiều nhãn trong một file procfs.

```
/ # cat /proc/lab2_info
LAB-02 Driver Statistics
=====================
Driver name : lab2
Major number : 240
Buffer size : 1024 bytes
Data length : 30 bytes
Open count : 18
Last data : [Embedded Linux Lab2 Test Data
]
/ # echo "===== SYSFS ATTRIBUTES ====="
===== SYSFS ATTRIBUTES =====
/ #
/ # SYSFS=/sys/class/lab2_class/lab2
/ #
/ # ls -la $SYSFS
lrwxrwxrwx    1 root     0                0 Oct 20 08:51 /sys/class/lab2_class/lab2 -> ../../devices/virtual/lab2_class/lab2
/ #
/ # echo "buffer_len = $(cat $SYSFS/buffer_len)"
buffer_len = 30
/ # echo "open_count = $(cat $SYSFS/open_count)"
open_count = 18
/ # echo "last_data  = $(cat $SYSFS/last_data)"
last_data  = Embedded Linux Lab2 Test Data
```

Hình 7. Ba file sysfs hiển thị buffer_len, open_count và last_data.

Đường dẫn	Mục đích	Triển khai
/dev/lab2	Trao đổi byte dữ liệu	lab2_fops: open/read/write/release
/proc/lab2_info	Đọc một báo cáo tổng hợp	seq_file: single_open + show
/sys/class/lab2_class/lab2/*	Đọc từng thuộc tính device	device_attribute, DEVICE_ATTR_RO

5. Phần C — MTD và JFFS2 trên Ubuntu HOST

5.1 Xác định NAND mô phỏng trước khi thao tác

Nhóm dùng nandsim để tạo thiết bị NAND raw. Ở máy của nhóm, /proc/mtd cho thấy “NAND simulator partition 0” tại mtd0 với 16 MiB dung lượng và 0x4000 (16 KiB) cho mỗi eraseblock. Đây là số đo trên

HOST của nhóm; không dùng thông số NAND 32 MiB hoặc tên thiết bị mtd1 được ghi trong tài liệu đối chiếu nếu máy không hiển thị như vậy.

```
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ sudo modprobe nandsim first_id_byte=0x20 second_id_byte=0x33
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ cat /proc/mtd
ls -l /dev/mtd*
dev:    size   erasesize  name
mtd0: 01000000 00004000 "NAND simulator partition 0"
crw------- 1 root root 90, 0 Thg 10 20 15:58 /dev/mtd0
crw------- 1 root root 90, 1 Thg 10 20 15:58 /dev/mtd0ro

/dev/mtd:
total 0
drwxr-xr-x 2 root root 60 Thg 10 20 15:58 by-name
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$
```

Hình 8. HOST: nandsim cung cấp thiết bị MTD và /proc/mtd.

5.2 Erase, tạo image và ghi vào NAND

Để tránh sự khác biệt giữa NAND và block device, nhóm thực hiện erase theo block trước khi nandwrite. mtdinfo xác nhận 1024 eraseblock × 16 KiB; image JFFS2 được tạo bằng mkfs.jffs2 -e 0x4000 và ghi vào /dev/mtd0 đã xác nhận là NAND mô phỏng.

```
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ sudo mtdinfo /dev/mtd0
mtd0
Name:                           NAND simulator partition 0
Type:                           nand
Eraseblock size:                16384 bytes, 16.0 KiB
Amount of eraseblocks:          1024 (16777216 bytes, 16.0 MiB)
Minimum input/output unit size: 512 bytes
Sub-page size:                  256 bytes
OOB size:                       16 bytes
Character device major/minor:   90:0
Bad blocks are allowed:         true
Device is writable:             true

letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ sudo flash_erase /dev/mtd0 0 0
Erasing 16384 Kibyte @ 0 -- 100 % complete
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$
```

Hình 9. Đo dung lượng NAND 16 MiB và hoàn tất flash_erase.

```
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ cd ~/embedded_lab2/embedded_lab2

mkdir -p jffs2_root/etc
printf "Embedded Linux Lab 2 - JFFS2\n" > jffs2_root/etc/lab2.txt

find jffs2_root -type f -o -type d

rm -f rootfs.jffs2

mkfs.jffs2 -d jffs2_root -o rootfs.jffs2 -e 0x4000 -n

ls -lh rootfs.jffs2
file rootfs.jffs2
jffs2_root
jffs2_root/data
jffs2_root/data/test.txt
jffs2_root/etc
jffs2_root/etc/lab2.txt
-rw-r--r-- 1 letienthanh letienthanh 512 Thg 10 20 16:01 rootfs.jffs2
rootfs.jffs2: Linux jffs2 filesystem data little endian
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ sudo nandwrite -p /dev/mtd0 rootfs.jffs2
Writing data to block 0 at offset 0x0
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$
```

Hình 10. Tạo rootfs.jffs2 và nạp qua nandwrite.

5.3 Mount, thay đổi dữ liệu và remount

Sau khi có /dev/mtdblock0, nhóm mount JFFS2 tới /mnt/jffs2 và đọc các file lab2.txt, test.txt. Nhóm tiếp tục chỉnh sửa nội dung, tạo runtime.txt, gọi sync và tháo/gắn lại filesystem. Những dữ liệu đã ghi vẫn xuất hiện khi mount lại: kết quả chứng minh tính duy trì dữ liệu qua remount trong cùng phiên nandsim, không phải thử nghiệm chống mất điện.

```
Try: sudo apt install <deb name>
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ sudo modprobe mtdblock
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ ls -l /dev/mtdblock*
brw-rw---- 1 root disk 31, 0 Thg 10 20 16:02 /dev/mtdblock0
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ sudo mount -t jffs2 /dev/mtdblock0 /mnt/jffs2
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ find /mnt/jffs2 -type f -o -type d
/mnt/jffs2
/mnt/jffs2/etc
/mnt/jffs2/etc/lab2.txt
/mnt/jffs2/data
/mnt/jffs2/data/test.txt
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ cat /mnt/jffs2/etc/lab2.txt
Embedded Linux Lab 2 - JFFS2
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ cat /mnt/jffs2/data/test.txt
Persistent data test
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ mount | grep jffs2
/dev/mtdblock0 on /mnt/jffs2 type jffs2 (rw,relatime)
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$
```

Hình 11. JFFS2 mount đọc/ghi được; nội dung file khớp dữ liệu mẫu.

```
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ sudo sh -c 'printf "Persistent after remount\n" >> /mnt/jffs2/data/test.txt'
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ sudo sh -c 'printf "Created during runtime\n" > /mnt/jffs2/runtime.txt'
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ cat /mnt/jffs2/data/test.txt
Persistent data test
Persistent after remount
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ cat /mnt/jffs2/runtime.txt
Created during runtime
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ sync
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ sudo umount /mnt/jffs2
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ sudo mount -t jffs2 /dev/mtdblock0 /mnt/jffs2
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ cat /mnt/jffs2/data/test.txt
Persistent data test
Persistent after remount
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ cat /mnt/jffs2/runtime.txt
Created during runtime
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ mount | grep jffs2
/dev/mtdblock0 on /mnt/jffs2 type jffs2 (rw,relatime)
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$
```

Hình 12. Nội dung dữ liệu tồn tại sau khi tháo và mount lại.

5.4 Phân tích raw NAND và cleanup

nanddump ở vùng đầu image cho byte 85 19, chính là magic 0x1985 của JFFS2 theo little-endian. Đây là bằng chứng ở tầng flash, bổ trợ cho việc mount thành công ở tầng filesystem. Kết thúc bài test, nhóm unmount và gỡ nandsim trên HOST.

```
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ sudo nanddump -o -l 512 -f /tmp/dump.bin /dev/mtd0
ECC failed: 0
ECC corrected: 0
Number of bad blocks: 0
Number of bbt blocks: 0
Block size 16384, page size 512, OOB size 16
Dumping data starting at 0x00000000 and ending at 0x00000200...
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ hexdump -C /tmp/dump.bin | head -20
00000000  85 19 01 e0 2c 00 00 00  5f 56 f1 e0 01 00 00 00  |....,...._V......|
00000010  00 00 00 00 02 00 00 00  93 2d d7 6a 04 04 00 00  |.........-.j....|
00000020  67 32 eb d2 7f 2c b7 8c  64 61 74 61 85 19 02 e0  |g2...,..data....|
00000030  44 00 00 00 1d fb f7 98  02 00 00 00 01 00 00 00  |D...............|
00000040  fd 41 00 00 e8 03 e8 03  00 00 00 00 93 2d d7 6a  |.A.............j|
00000050  93 2d d7 6a 93 2d d7 6a  00 00 00 00 00 00 00 00  |.-.j.-.j........|
00000060  00 00 00 00 00 00 00 00  00 00 00 00 47 7c 5a d2  |............G|Z.|
00000070  85 19 01 e0 2b 00 00 00  e6 6e 26 7d 01 00 00 00  |....+....n&}....|
00000080  01 00 00 00 03 00 00 00  db 2d d7 6a 03 04 00 00  |.........-.j....|
00000090  78 0e 89 80 db 85 f4 d1  65 74 63 ff 85 19 02 e0  |x.......etc.....|
000000a0  44 00 00 00 1d fb f7 98  03 00 00 00 01 00 00 00  |D...............|
000000b0  fd 41 00 00 e8 03 e8 03  00 00 00 00 db 2d d7 6a  |.A.............-.j|
000000c0  db 2d d7 6a db 2d d7 6a  00 00 00 00 00 00 00 00  |.-.j.-.j........|
000000d0  00 00 00 00 00 00 00 00  00 00 00 00 9c 8f c9 d3  |................|
000000e0  85 19 01 e0 30 00 00 00  78 be 3e fa 02 00 00 00  |....0...x.>.....|
000000f0  02 00 00 00 04 00 00 00  93 2d d7 6a 08 08 00 00  |.........-.j....|
00000100  f2 b9 13 56 12 9b eb d3  74 65 73 74 2e 74 78 74  |...V....test.txt|
00000110  85 19 02 e0 59 00 00 00  5f 74 84 3a 04 00 00 00  |....Y..._t.:....|
00000120  01 00 00 00 b4 81 00 00  e8 03 e8 03 15 00 00 00  |................|
00000130  93 2d d7 6a 93 2d d7 6a  93 2d d7 6a 00 00 00 00  |.-.j.-.j.-.j....|
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$
```

Hình 13. Raw dump cho biết JFFS2 magic 85 19; thống kê ECC/bad block.

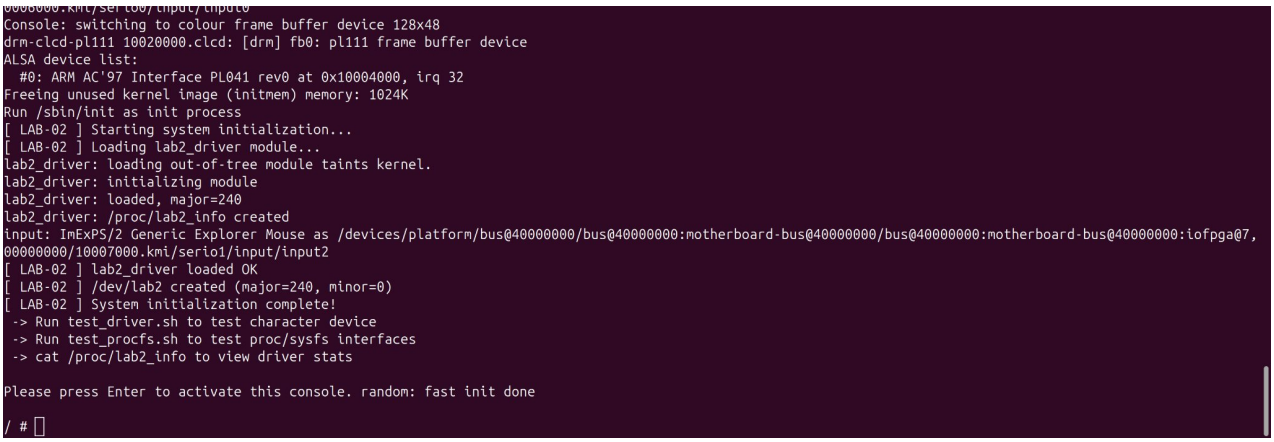
```
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ sudo umount /mnt/jffs2
sudo modprobe -r mtdblock
sudo modprobe -r nandsim
ls /dev/mtd0 2>/dev/null || printf 'MTD device removed\n'
uname -r
[sudo] password for letienthanh:
MTD device removed
6.8.0-147-generic
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$ ls /dev/mtd0 2>/dev/null || printf 'MTD device removed\n'
uname -r
MTD device removed
6.8.0-147-generic
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab2/embedded_lab2$
```

Hình 14. Cleanup HOST sau thực hành; kiểm tra kernel host 6.8.0-147-generic.

6. Phần D — BusyBox init tự nạp driver

Khi Kernel chạy /sbin/init, BusyBox init tra /etc/inittab, thực thi /etc/init.d/rcS theo action sysinit, sau đó tạo console shell. rcS gắn các filesystem ảo, gọi insmod với lab2_driver.ko và tạo /dev/lab2 khi tải module thành công. Archive initramfs_lab2.cpio.gz chứa module cùng hai script test_driver.sh và test_procfs.sh.

```
/etc/inittab → ::sysinit:/etc/init.d/rcS
rcS → mount proc/sysfs → insmod lab2_driver.ko
    → /dev/lab2 (240:0) → BusyBox console → test scripts
```



Hình 15. QEMU TARGET: rcS nạp driver tự động trong giai đoạn boot.



Hình 16. Archive initramfs có module và script test.

Hai script được báo PASS trong ảnh kiểm chứng; việc tự tải module trước khi chạy script có thể làm insmod lặp. Script kiểm tra lsmod để chỉ nạp khi module chưa có sẽ tránh lỗi “File exists”.

7. Đối chiếu các filesystem phổ biến

Loại FS	Ghi dữ liệu	Đặc trưng khi dùng flash	Ứng dụng phù hợp
ext4	Có	Journal trên block device; nhờ FTL nếu lưu trên eMMC/SD	eMMC, SD, SSD
JFFS2	Có	Log-structured, garbage collection; mount quét node	Raw NAND/NOR nhỏ
cramfs	Không	Nén, image chỉ đọc	Firmware đơn giản
UBIFS	Có	Đi qua UBI để quản lý eraseblock/bad block	Raw NAND cỡ vừa-lớn
SquashFS	Không	Nén cao; image bất biến	Rootfs chỉ đọc

Phương án cho IoT có 32 MiB NAND và cần OTA: ưu tiên phân vùng rootfs có cơ chế xác thực, cập nhật dự phòng A/B và rollback; có thể kết hợp UBI/UBIFS cho phần dữ liệu thay đổi nếu giới hạn thiết bị cho phép. JFFS2 phù hợp dữ liệu dung lượng nhỏ nhưng thời gian mount có thể tăng khi vùng flash lớn dần. Đây là đề xuất thiết kế, chưa phải hệ thống OTA được nhóm xây dựng hay kiểm thử trong Lab 2.

8. Trả lời năm câu hỏi phân tích

Câu 1 Vì sao phải dùng copy_to_user/copy_from_user?

Con trỏ tới userspace có thể trỏ tới vùng chưa map, bị thay đổi hoặc không có quyền truy cập. Các API uaccess của kernel có cơ chế truy cập và xử lý fault dành cho địa chỉ user; kết quả trả về là số byte không

copy được. Driver thường chuyển tình huống không sao chép được thành -EFAULT. memcpy thông thường không cung cấp hợp đồng bảo vệ tương đương đối với con trỏ user do tiến trình đưa vào.

Câu 2 Mutex giải quyết race condition như thế nào?

device_buffer và buffer_len thuộc trạng thái dùng chung. Nếu hai tiến trình ghi xen kẽ, dữ liệu và chiều dài có thể không tương ứng, đặc biệt khi copy_from_user() có khả năng sleep. Mutex bao vùng cập nhật giúp tránh ghi đồng thời; để có snapshot nhất quán, read() và các hàm show() cũng phải được xem xét cùng cơ chế đồng bộ. Test tuần tự trong báo cáo chưa chứng minh tính đúng đắn dưới tải concurrent.

Câu 3 procfs khác sysfs ở điểm nào?

procfs thường xuất thông tin tiến trình/hệ thống hoặc thống kê tổng hợp với định dạng tự do; sysfs được tổ chức theo device model và kobject, khuyến khích thuộc tính có ngữ nghĩa ổn định, thường mỗi thuộc tính một file. Trong bài, /proc/lab2_info dùng seq_file, còn sysfs có buffer_len, open_count và last_data được công bố qua các hàm show read-only.

Câu 4 Vì sao NAND không sửa trực tiếp giống ổ đĩa?

Cell flash thường chuyển 1→0 khi program; khôi phục 0→1 đòi hỏi erase cả eraseblock. Ở thiết bị giả lập của nhóm, page 512 byte trong khi eraseblock 16 KiB. JFFS2 ghi node mới, đánh dấu phiên bản cũ obsolete rồi thu gom rác/erase block khi cần. ext4 trên thiết bị block thường dựa vào tầng điều khiển flash để che giấu việc erase.

Câu 5 Nếu insmod trong rcS thất bại thì sao?

rcS là một shell script. Khi insmod trả lỗi, hệ thống có tiếp tục boot hay không phụ thuộc cách script xử lý exit code; lỗi riêng của insmod không tự tạo kernel panic. Nếu driver không bắt buộc, script nên thông báo lỗi rồi đưa máy vào chế độ hạn chế; nếu driver thiết yếu, sản phẩm cần cơ chế khôi phục hoặc chuyển sang firmware dự phòng. Trong ảnh thực nghiệm của nhóm, nhánh nạp module đã thành công.

9. Khó khăn, an toàn thao tác và kết luận

Vấn đề	Giải pháp / kinh nghiệm
/proc/mtd chưa hiển thị NAND	Kiểm tra module nandsim đã nạp và đọc nhận dạng thiết bị trước thao tác.
Lệnh dán bị chèn ký tự lạ	Chạy lại từng dòng sạch; không dùng kết quả bị lỗi làm minh chứng.
Thư mục dữ liệu JFFS2 thiếu trước khi đóng gói	Tạo đủ cây mẫu và kiểm tra find trước mkfs.jffs2.
Ví dụ 32 MiB không khớp NAND 16 MiB của nhóm	Ghi đúng số đo mtdinfo thay vì sử dụng thông số tham khảo.
Nguy cơ xóa nhầm BIOS firmware của HOST	Chỉ flash_erase trên MTD có tên NAND simulator, tuyệt đối không suy diễn từ mtd index.

Kết luận: Lab 2 cho thấy đường đi từ lời gọi userspace tới kernel driver, các giao diện quan sát trạng thái và cơ chế tự nạp module. Trên HOST, nandsim/JFFS2 đã chứng minh thao tác erase, ghi image, mount và remount. Các kết quả là bằng chứng chức năng trong phạm vi bài thực nghiệm; chưa mở rộng tới stress race condition, tắt nguồn đột ngột hay OTA thực tế.

Tài liệu đối chiếu: Đề LAB-02 môn Embedded Linux Systems; Lab02_Report.pdf do người dùng cung cấp (dùng để định hướng phạm vi, không sử dụng kết quả thí nghiệm của nhóm khác); ảnh/log từ báo cáo Lab 2 của nhóm trên Ubuntu HOST và QEMU TARGET.